深度学习基础

第 4 讲 — 损失函数、训练技巧与优化策略

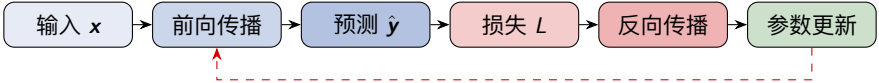
孟翰

人工智能学院
中国石油大学（北京）

2026 年春季学期

本讲内容

1. 回顾与引入
2. 损失函数与输出层设计
3. 参数初始化
4. 批量归一化
5. 正则化：防止过拟合
6. 优化器：从 SGD 到 Adam
7. 学习率调度
8. 总结与标准配置



已掌握的核心知识

- MLP 结构与前向传播：$\boldsymbol{a}^{(l)} = f(\boldsymbol{W}^{(l)}\boldsymbol{a}^{(l-1)} + \boldsymbol{b}^{(l)})$
- 反向传播四公式：$\delta^{(L)} = \hat{\boldsymbol{y}} - \boldsymbol{y}$，$\delta^{(l)} = (\boldsymbol{W}^{(l+1)})^\top \delta^{(l+1)} \odot f'(\boldsymbol{z}^{(l)})$
- 激活函数：ReLU 是隐藏层默认选择

从“会算梯度”到“能训好模型”，还差什么？

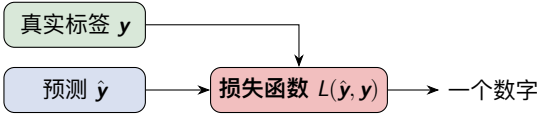
1. **损失函数**：上次只讲了交叉熵，回归任务用什么？分类**为什么不能用 MSE**？
2. **初始化**：参数随便取行不行？怎样才能让梯度稳定传播？
3. **过拟合**：网络能拟合训练集，但测试集表现差怎么办？
4. **优化器**：朴素梯度下降太慢太曲折，有没有更好的方法？

本讲目标

掌握从**拿到任务**到**训练出好模型**的完整实践流程。

回顾：训练的目标

上次课我们学了反向传播——它能计算每个参数的梯度。但梯度是“损失对参数的导数”，**损失函数**就是告诉网络“你错了多少”的**评分标准**。



选错损失函数会怎样？

- 用 MSE 做分类 → 梯度信号微弱，训练极慢甚至不收敛
- 用交叉熵做回归 → 数学上不合理，输出无意义

损失函数的选择直接决定了**网络能不能学会**，不同任务需要不同的损失函数。

神经网络的最后一层决定了输出的含义

网络最后一层（输出层）的**激活函数**决定了输出值的范围和解释方式：

隐藏层输出 → 线性变换 $\xrightarrow{\text{原始分数}}$ 输出层激活 → 最终输出

不同任务需要不同的输出设计

- **回归**（预测房价）：输出可以是任意实数 $\rightarrow$ 不加激活函数
- **二分类**（垃圾邮件）：输出需要是概率 $\in (0, 1)$ $\rightarrow$ Sigmoid
- **多分类**（图像识别）：输出需要是概率分布 $\rightarrow$ Softmax

核心原则

输出层激活函数和**损失函数**必须**配套使用**，下面我们逐一讲解。

MSE（最常用）

$$L = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- 对大误差惩罚重
- 对异常值**敏感**

MAE

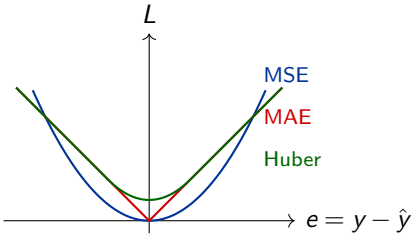
$$L = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

- 对异常值**鲁棒**
- 零点不可导

Huber 损失

$$L_\delta = \begin{cases} \frac{1}{2}e^2, & |e| \le \delta \\ \delta|e| - \frac{\delta^2}{2}, & |e| > \delta \end{cases}$$

- 小误差用 MSE
- 大误差用 MAE



MSE 用于分类的问题

Sigmoid 输出接近 0 或 1 时，$\sigma'(z) \approx 0$

MSE 梯度：

$$\frac{\partial L}{\partial z} = (\hat{y} - y) \cdot \sigma'(z)$$

- 在饱和区，即使误差很大，**梯度也几乎为零**
- 预测完全错误时，学习速度**反而最慢**！

交叉熵的优势

交叉熵梯度：

$$\frac{\partial L}{\partial z} = \hat{y} - y$$

- 不受 Sigmoid 饱和影响
- 误差越大，**梯度越大**
- 学习速度与误差成正比
- 有信息论基础（KL 散度）

铁律

分类问题**永远**使用交叉熵损失，不要使用 MSE！

任务类型	输出层激活	损失函数	输出解释
回归	无（线性）	MSE	预测数值
回归（有异常值）	无（线性）	Huber	预测数值
二分类	Sigmoid	BCE	$P(y=1\|\boldsymbol{x})$
多分类（互斥）	Softmax	CE	各类别概率
多标签（共存）	Sigmoid	BCE	各标签独立概率

多分类 vs 多标签——学生常混淆

- **多分类**（图像分类：猫/狗/鸟）：类别**互斥**，Softmax 输出和为 1
- **多标签**（图像标注：有人、有车、有树）：标签可**共存**，每个输出独立 Sigmoid

三种常见配对，输出层误差形式完全相同： $\boxed{\boldsymbol{\delta}^{(L)} = \hat{\boldsymbol{y}} - \boldsymbol{y}}$

MSE + 线性输出

$$L = \frac{1}{2}\|\boldsymbol{a}^{(L)} - \boldsymbol{y}\|^2 \;\Rightarrow\; \boldsymbol{\delta}^{(L)} = \boldsymbol{a}^{(L)} - \boldsymbol{y}$$

交叉熵 + Softmax

$$L = -\sum_k y_k \log a_k^{(L)} \;\Rightarrow\; \boldsymbol{\delta}^{(L)} = \boldsymbol{a}^{(L)} - \boldsymbol{y}$$

这不是巧合

损失函数与输出层激活函数**精心配对**的结果——梯度形式简洁，误差信号直接反映“预测与真实的差距”。实践中请**永远使用标准配对**。

回顾：网络的参数从哪里来?

我们设计好了网络结构（多少层、每层多少神经元），但里面的**权重** W 和**偏置** b 还没有值。训练**开始之前**，必须给它们赋一个初始值——这就是**参数初始化**。

类比

参数初始化就像**考试前选择座位**：

- 坐在好位置（好的初始化）→ 看得清黑板，答题顺利
- 坐在角落（差的初始化）→ 看不清，从一开始就吃亏
- 所有人挤在同一个位置（零初始化）→ 完全没法学习

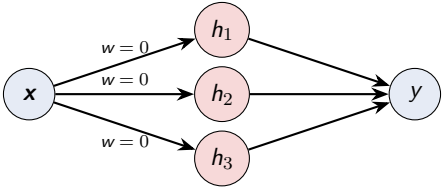
为什么初始化这么重要?

初始化决定了训练**第一步**的梯度大小和方向。如果初始化不好，梯度从一开始就会消失或爆炸，网络**根本无法训练**。

对称性问题

若所有权重初始化为 0，同一层所有神经元将：

1. 接收**相同输入** → 产生**相同输出** → 获得**相同梯度** → 进行**相同更新**



$h_1 \equiv h_2 \equiv h_3$，3 个神经元等价于**只有 1 个**！

结论

零初始化完全浪费了网络的宽度。**必须**用随机初始化打破对称性——但方差怎么选？

从 $\mathcal{N}(0, \sigma^2)$ 采样打破对称性，但 σ 至关重要

σ 太大

- 输出值很大，激活函数饱和
- Sigmoid/Tanh 进入平坦区
- 梯度消失，训练一开始就**卡住**

σ 太小

- 输出值集中在零附近
- 信号逐层衰减
- 深层几乎**收不到信息**

理想目标

选择 σ 使每层输出的**方差保持稳定**——信号既不放大也不缩小，安全地穿过所有层。

Xavier / Glorot 初始化

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right)$$

- 兼顾前向和反向传播的方差稳定
- 适用于 **Sigmoid / Tanh**

He / Kaiming 初始化

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right)$$

- ReLU 砍掉约一半信号，方差**加倍**补偿
- 适用于 **ReLU 及其变体**
- **现代网络首选**

初始化	方差	适用激活	推荐度
零初始化	0	—	禁止
Xavier	$\frac{2}{n_{\text{in}}+n_{\text{out}}}$	Sigmoid/Tanh	经典
He	$\frac{2}{n_{\text{in}}}$	ReLU 系列	首选

偏置 $\boldsymbol{b}$ 通常初始化为**零向量**。PyTorch 中 `nn.Linear` 默认使用 Kaiming 均匀初始化。

过渡：从初始化到归一化

好的初始化让训练**起步顺利**，但随着训练进行，每一层的输入分布会不断变化——初始化的好处会逐渐消失。我们需要一种方法，让每一层的输入**始终保持稳定**。

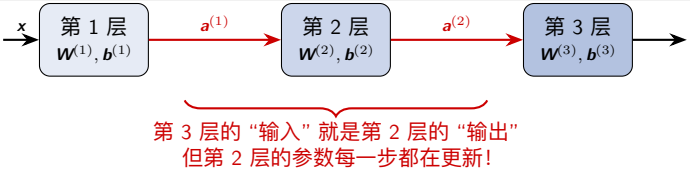
批量归一化（Batch Normalization）是什么？

一句话：在网络的每一层中间插入一个“标准化”操作，把该层的输入**拉回**到均值为 0、方差为 1 的标准分布。

BN 在网络中的位置

BN 像一个“稳压器”，插在每层的线性变换和激活函数之间。

关键观察：第 3 层的输入 = 第 2 层的输出

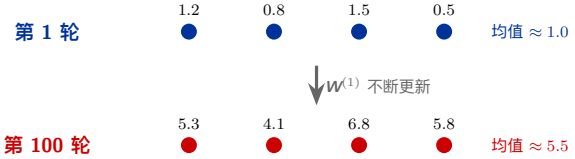


连锁反应

1. 第 1 步训练：梯度下降更新了 $W^{(1)}$
2. $W^{(1)}$ 一变 $\rightarrow$ 第 1 层的输出 $a^{(1)}$ 的分布就变了
3. $a^{(1)}$ 变了 $\rightarrow$ 第 2 层收到的输入变了 $\rightarrow$ 第 2 层的输出 $a^{(2)}$ 也变了
4. $a^{(2)}$ 变了 $\rightarrow$ 第 3 层收到的输入**又变了**……

每更新一次参数，后面所有层的输入分布都会跟着变！

假设第 2 层某个神经元，对同样的 4 个训练样本输出的值：



第 3 层在第 1 轮看到的输入在 $[0.5, 1.5]$，第 100 轮变成 $[4.1, 6.8]$——**完全不同的数值范围**！

投篮比喻

你在练投篮，但**每投一次，篮筐就移动一下**——永远在追赶目标，很难提高。

BN 的解决方案

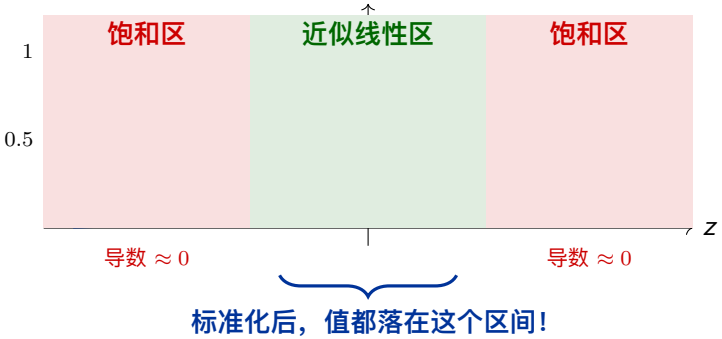
不管前面的层怎么变，BN 都把值**拉回均值 0、方差 1**——相当于**把篮筐固定住**！

场景：某一层的某个神经元，一个批次有 4 个样本

原始值：	5.2	3.8	6.0	5.0	**Step 1**: $\mu = \frac{5.2+3.8+6.0+5.0}{4} = 5.0$ **Step 2**: $\sigma^2 = 0.605$，$\sigma \approx 0.78$
标准化：	0.26	−1.54	1.28	0.0	**Step 3**: $\hat{x}_i = \frac{x_i - 5.0}{0.78}$ 现在：均值 $= 0$，方差 $= 1$
缩放平移：	y_1	y_2	y_3	y_4	**Step 4**: $y_i = \gamma \hat{x}_i + \beta$ γ, β 由网络**自动学习**

关键点

- Step 1–3：强制把分布**拉回**均值 0、方差 1（“稳压”）
- Step 4：用可学习的 γ, β 让网络自己决定**最终分布**——不会限制表达能力



问题

标准化把值集中到均值 0、方差 1（大约在 $[-1, 1]$），正好是 Sigmoid 的**近似线性区**。在线性区里，$\sigma(z) \approx 0.5 + 0.25z$——**线性函数**！这样 Sigmoid 就丧失了非线性能力，多层网络又退化为单层。

Step 4: $y_i = \gamma \hat{x}_i + \beta$（$\gamma, \beta$ 由网络自动学习）

γ 控制分布的**宽窄**，β 控制分布的**位置**——网络可以把值移到 Sigmoid 的任意区域。

只做标准化
值都在 $[-1, 1]$
困在线性区，丢失非线性

$\xrightarrow{\text{加上 } \gamma, \beta}$

标准化 + γ, β
网络自动选择分布位置
稳定又保留表达能力

网络可以学到三种策略

- 需要非线性 → 学到合适的 γ, β，把分布移到 Sigmoid 的**弯曲区域**
- 标准化后刚好 → 学到 $\gamma = 1, \beta = 0$，**保持**标准化结果
- 不想要标准化 → 学到 $\gamma = \sigma, \beta = \mu$，**完全恢复**原始分布

结论

γ, β 保证 BN 只会**帮忙**，不会**帮倒忙**——最差也就是恢复到没有 BN 的状态。

1. 加速收敛：训练速度提升数倍	每层输入稳定，参数更新更高效
2. 允许更大学习率：梯度更稳定	不用担心梯度爆炸，可以大胆用大学习率
3. 降低初始化敏感性	BN 会“纠正”不好的初始值
4. 轻微正则化效果	批统计量有随机性，类似 Dropout 的噪声

BN 为什么几乎是标配?

一个组件同时解决了训练稳定性、收敛速度、初始化敏感性三大问题——性价比极高。

训练时

用**当前批次**的 $\mu_{\mathcal{B}}$，$\sigma_{\mathcal{B}}^2$
同时维护**全局滑动平均**：
$\mu_{\text{run}} \leftarrow 0.9\mu_{\text{run}} + 0.1\mu_{\mathcal{B}}$

部署 →

推理时

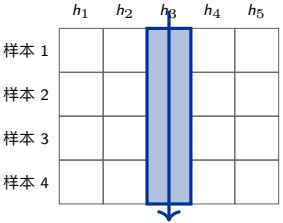
用训练时积累的
全局滑动平均 μ_{run}，σ_{run}^2
因为推理时可能只有 **1** 个样本，
没有“批次”可以算统计量！

最常见的 Bug——每年都有同学犯！

忘记在推理前切换模式，导致 BN 用的是不稳定的批统计量，**预测结果忽高忽低**！

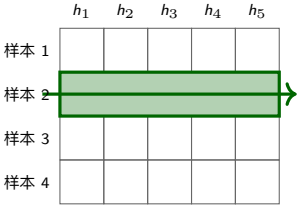
训练时	`model.train()` ——BN 使用批统计量
推理时	`model.eval()` ——BN 使用全局滑动平均

BatchNorm：竖着归一化



对 **同一特征** h_3
取 4 个样本的均值方差

LayerNorm：横着归一化



对**同一样本**（样本 2）
取 5 个特征的均值方差

	归一化方向	依赖批大小?	适用场景
BatchNorm	同一特征，跨样本（竖向）	是（太小不准）	**CNN**
LayerNorm	同一样本，跨特征（横向）	否	**Transformer**

假设一个批次有 3 个样本，每个样本有 4 个特征

	h_1	h_2	h_3	h_4
样本 A	1	3	5	7
样本 B	2	4	6	8
样本 C	3	5	7	9

：对 h_1 这一列（竖向）

值：$\{1,2,3\} \to \mu=2$，$\sigma\approx0.82$
再对 h_2, h_3, h_4 各列分别算。
需要多个样本才能算！

LN：对样本 A 这一行（横向）

值：$\{1,3,5,7\} \to \mu=4$，$\sigma\approx2.24$
再对样本 B、C 各行分别算。
只看自己，不依赖其他样本！

```python
import torch.nn as nn

# 在网络中使用 BN：放在激活函数之前
model = nn.Sequential(
    nn.Linear(784, 256),
    nn.BatchNorm1d(256),  # BN 层
    nn.ReLU(),
    nn.Linear(256, 128),
    nn.BatchNorm1d(128),  # BN 层
    nn.ReLU(),
    nn.Linear(128, 10),
)

# 训练时
model.train()   # 使用批统计量
output = model(x_train)

# 推理时
model.eval()    # 使用全局滑动平均
output = model(x_test)
```

过渡：从“训不动”到“训过头”

前面我们解决了训练稳定性的问题（初始化 + BN）。现在假设网络能正常训练了，但出现了一个新问题：

训练集上准确率 99%，测试集上只有 70%——怎么回事？

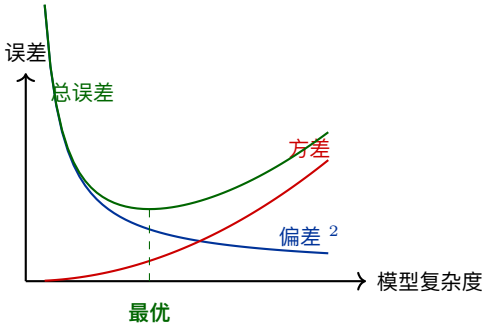
这就是**过拟合**（Overfitting）

模型不是在学习“规律”，而是在**死记硬背**训练数据（包括噪声和偶然性），导致面对新数据时表现很差。

上次课讲了偏差-方差分解：过拟合 = 高方差 = 模型太复杂。

什么是正则化？

正则化是一类**限制模型复杂度**的技术，目的是让模型学到**真正的规律**而非训练集的噪声。接下来介绍三种最常用的方法。



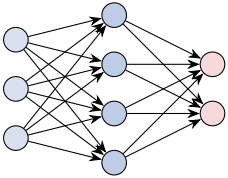
深度网络的现实

- 参数量远大于样本量
- 极强的拟合能力
- 容易“记住”训练集噪声
- **过拟合几乎必然发生**

三种实用解决方案

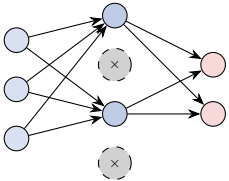
Dropout（随机失活）、**L2 正则化**（权重衰减）、**早停法**（验证集监控）

标准网络



4 个神经元全部工作

训练时加了 Dropout



随机关掉 2 个，只剩 2 个工作

核心思想

训练时，每次前向传播**随机**以概率 $1-p$ 关闭一些神经元（输出置为 0），以概率 p 保留。

- 常用设置：隐藏层 $p=0.5$（一半关掉），输入层 $p=0.8$
- 每次前向传播**随机关不同的**神经元 $\rightarrow$ 相当于每次训练一个**不同的**子网络

隐藏层有 4 个神经元，$p = 0.5$（保留概率 50%），激活值为：

原始激活值：	2.0	−1.5	3.0	0.8	
随机掩码：	1	0	1	0	（随机生成）
Dropout 后：	4.0	0	6.0	0	关掉的输出 0 保留的 $\div p = \div 0.5$

关键操作：保留的值要除以 p（Inverted Dropout）

- 原来 $h_1 = 2.0$，Dropout 后变成 $2.0/0.5 = 4.0$
- 为什么？因为只有一半神经元在工作，放大 $1/p$ 倍让**总输出的期望不变**
- 期望验证：$E[\tilde{h}] = p \cdot (h/p) + (1-p) \cdot 0 = h$ ✓

训练时（model.train()）

随机关闭一部分神经元
保留的值**除以** p
每次前向传播是**不同的子网络**

推理时（model.eval()）

所有神经元都工作
输出**不做任何调整**
（因为训练时已经除以 p 了）

和 BN 一样，必须切换模式！

如果推理时忘了 `model.eval()`，Dropout 仍然会随机关闭神经元——每次推理同一张图片得到**不同的结果**！

	BN 的行为	**Dropout 的行为**
`model.train()`	用批统计量	**随机关闭**神经元
`model.eval()`	用全局滑动平均	**全部打开**，不做丢弃

角度 1：集成学习

- n 个神经元 $\rightarrow 2^n$ 种子网络
- 每次训练一个不同的子网络
- 推理时用完整网络 $\approx$ 所有子网络的**平均预测**
- 集成学习通常比单模型更好

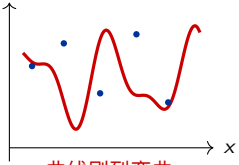
角度 2：去除共适应

- 神经元不能依赖特定伙伴
- 每个神经元必须**独立有用**
- 学到的特征更鲁棒、更泛化

类比

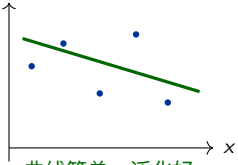
团队中每次随机有人请假 $\rightarrow$ 每个人学会**独当一面**

权重大 → 过拟合



曲线剧烈弯曲

权重小 → 平滑泛化好



曲线简单，泛化好

过拟合的特征

过拟合的模型往往学出**很大的权重**——对输入的微小变化产生剧烈反应，曲线极其弯曲。

L2 正则化的思路

惩罚大权重！在损失函数里加一项：权重越大，惩罚越大 → 迫使网络用**小权重**拟合数据 → 曲线更平滑。

在损失函数后面加上权重的平方和

$$L_{\text{reg}} = \underbrace{L}_{\text{原始损失}} + \underbrace{\frac{\lambda}{2}\|\boldsymbol{w}\|^2}_{\text{惩罚项}}, \qquad \lambda > 0$$

λ 越大，对大权重的惩罚越强；$\lambda = 0$ 就是没有正则化。

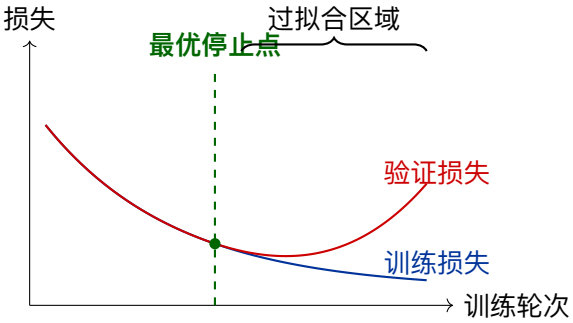
权重更新公式——为什么叫 “权重衰减”？

$$w \leftarrow \underbrace{(1-\eta\lambda)}_{\text{衰减因子}<1} w - \eta \frac{\partial L}{\partial w}$$

每次更新前权重先乘 $(1-\eta\lambda) < 1$，被**均匀缩小**。

实践

典型值 $\lambda = 10^{-4} \sim 10^{-2}$。PyTorch 一行搞定：`Adam(params, lr=1e-3, weight_decay=1e-4)`



策略

1. 训练时监控**验证损失**
2. 验证损失连续 **patience** 轮不降 → 停止
3. 保存验证损失**最低时**的参数

优势

- 最**简单**的正则化
- **零额外计算**开销
- 自动确定训练时长
- patience 通常设 5∼20

回顾：参数怎么更新?

上次课我们学了反向传播，算出了每个参数的梯度。然后用最朴素的方法更新：

$$\text{新参数} = \text{旧参数} - \text{学习率} \times \text{梯度}$$

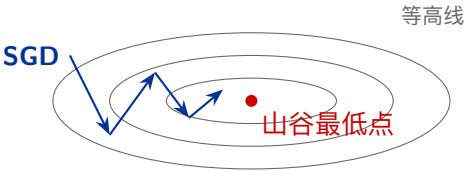
这就是最基础的**优化器**——它决定了“拿到梯度之后，参数**怎么走一步**”。

类比：下山

你站在雾蒙蒙的山上，看不到全貌，只能感受到**脚下的坡度**（= 梯度）。

优化器就是你的下山策略：

- **朴素 SGD**：每一步都只看脚下坡度，直接往最陡的方向走
- **动量法**：记住之前走的方向，有**惯性**，不会被小坑绊住
- **Adam**：不仅有惯性，还会根据地形**自动调整步幅**——陡坡小步走，平地大步走



想象一个狭长的山谷

- 山谷**两侧很陡**（上下方向梯度大）
- 山谷**底部很平缓**（左右方向梯度小）
- SGD 每一步只看脚下坡度
- 结果：上下**来回弹跳**，左右进展**极慢**

SGD 的两个问题

1. **没有记忆**：只看当前梯度，路径曲折
2. **步幅统一**：所有方向一刀切

核心想法：不只看当前梯度，把之前几步的方向也考虑进来

连续 5 步的梯度

步数	上下方向	左右方向
1	$+3$	$+0.5$
2	-2	$+0.4$
3	$+4$	$+0.6$
4	-3	$+0.5$
5	$+2$	$+0.5$
累积	≈ 0（抵消）	≈ 2.5（叠加）

效果

- 上下方向正负交替 → 动量**抵消** → 震荡减小
- 左右方向一直同向 → 动量**叠加** → 加速前进

公式

$$\boldsymbol{v}_t = 0.9\,\boldsymbol{v}_{t-1} + \eta \nabla L$$
$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \boldsymbol{v}_t$$

$0.9 =$ 记住 90% 的历史

动量法解决了“曲折”问题，但还有一个问题没解决

参数 A：梯度 ± 100

$\eta = 0.01$ 时一步走 1.0
步子太大，**震荡发散**！

参数 B：梯度 ± 0.01

$\eta = 0.01$ 时一步走 0.0001
步子太小，**几乎不动**！

Adam 的思路：因人而异

给每个参数**单独**自动调整学习率——梯度大的用小步，梯度小的用大步，不再一刀切。

Adam = 动量（稳定方向）+ 自适应学习率（因人而异）

Adam 做了两件事

1. **记住梯度的方向**（m）：取梯度的加权平均，和动量法一样

$$m_t = 0.9\, m_{t-1} + 0.1\, \nabla L$$

2. **记住梯度的大小**（v）：追踪梯度**有多剧烈**

$$v_t = 0.999\, v_{t-1} + 0.001\, (\nabla L)^2$$

然后用 v 来自动调整步幅：$\theta \leftarrow \theta - \frac{\eta}{\sqrt{v_t}+\epsilon} \cdot m_t$

- v_t 大（梯度剧烈）→ 分母大 → 步子**自动变小**（谨慎走）
- v_t 小（梯度平稳）→ 分母小 → 步子**自动变大**（大胆走）

Adam 完整公式（供参考）

$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla L$，　$v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla L)^2$，　偏差修正后更新 $\theta \leftarrow \theta - \frac{\eta}{\sqrt{\hat{v}_t}+\epsilon}\hat{m}_t$

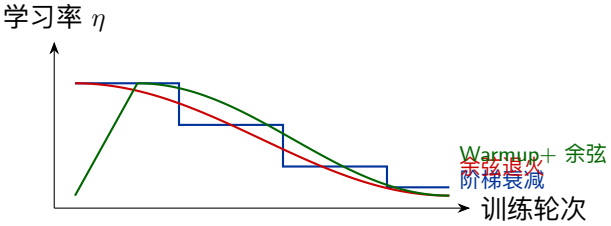
	SGD	SGD+ 动量	Adam
记住历史方向	否	是	是
自适应步幅	否	否	是
调参难度	难	中	**简单**

实践建议

- 默认超参数 $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 10^{-3}$——**通常不需要调**
- 初学者**先用 Adam**；NLP/Transformer 用 AdamW；图像精调可尝试 SGD+ 动量

为什么不用固定学习率?

训练**初期**需要大步长快速接近最优区域，**后期**需要小步长精细调整——固定学习率无法兼顾。



Warmup + 余弦退火 = 现代标准配置

- **Warmup**（前 5%~10% 轮次）：η 从 0 线性增长到 $\eta_{\max}$，避免初期发散
- **余弦退火**：$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos\frac{t}{T}\pi)$，平滑下降到 $\eta_{\min}$

训练稳定
- He 初始化
- 批量归一化

防止过拟合
- Dropout
- L2 正则化
- 早停法

优化加速
- 动量法
- Adam 优化器
- 学习率调度

现代深度学习 “标准配置”

He 初始化 + BatchNorm + ReLU + Dropout + Adam ($\eta = 10^{-3}$) + Warmup 余弦退火

这套组合能应对大多数任务！

从任务到训好模型的 6 步

1. **确定任务类型** → 选输出层激活 + 损失函数（回归 →MSE，分类 →CE）
2. **搭建网络** → He 初始化 + BN + ReLU
3. **选择优化器** → Adam, $\eta = 10^{-3}$, weight_decay $= 10^{-4}$
4. **加正则化** → Dropout 0.5 + 早停 patience=10
5. **训练** → 观察训练/验证曲线
6. **调优** → 根据曲线判断欠拟合/过拟合，调整容量或正则化

诊断指南

症状	对策
训练损失不下降	增大网络 / 增大学习率 / 检查数据
训练损失低但验证损失高	加 Dropout / 加 L2 / 早停 / 加数据
训练不稳定（NaN/震荡）	减小学习率 / 加 BN / 检查初始化

```python
import torch.nn as nn
# Step 1: 搭建网络
model = nn.Sequential(
    nn.Linear(784, 256), nn.BatchNorm1d(256), nn.ReLU(), nn.Dropout(0.5),
    nn.Linear(256, 128), nn.BatchNorm1d(128), nn.ReLU(), nn.Dropout(0.5),
    nn.Linear(128, 10))
# Step 2: 选择损失函数和优化器
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)
# Step 3: 训练循环
model.train()
for epoch in range(num_epochs):
    output = model(x_train)            # 前向传播
    loss = criterion(output, y_train)  # 计算损失
    loss.backward()                    # 反向传播
    optimizer.step()                   # 参数更新
    optimizer.zero_grad()              # 清零梯度
# Step 4: 推理（注意切换模式!）
model.eval()
with torch.no_grad():
    pred = model(x_test)
```

第 5 讲：卷积神经网络基础

全连接网络处理图像时参数**爆炸**——$224 \times 224 \times 3$ 的图像仅第一层就需要 1.5 亿参数！

如何利用图像的**局部性**和**平移不变性**?

- 卷积操作：局部连接 + 参数共享
- 填充、步幅与输出尺寸
- 池化层：降维与不变性
- CNN 基本结构与 LeNet

思考题

1. 一个图像标注任务：给定一张图，需要输出 “有人”“有车”“有树” 三个标签（可同时存在）。应该用什么输出层激活函数和损失函数？为什么不能用 Softmax？
2. 如果训练中发现训练损失很低但验证损失很高，你会按什么顺序尝试哪些方法？
3. 为什么 BN 在训练和推理时行为不同？如果推理时忘记调用 `model.eval()` 会发生什么？
4. Adam 的默认学习率是 10^{-3}，SGD 的典型学习率是 $10^{-1} \sim 10^{-2}$。为什么 Adam 可以用更小的学习率？

谢谢！

孟翰
人工智能学院
中国石油大学（北京）

hmeng@cup.edu.cn